

Assignment – 2

1.Create a custom hook called useCurrentTime that returns the current time and updates every second, then use it to display a live clock in a functional component.

Answer:-

useCurrent.js

```
import React, { useEffect, useState } from 'react'

const useCurrent = () => {
  const [currentTime, setCurrentTime] = useState(
    new Date().toLocaleTimeString()
  );
  useEffect(() => {
    const timer = setInterval(() => {
      setCurrentTime(new Date().toLocaleTimeString());
    }, 1000);
    return () => clearInterval(timer);
  }, []);

  return (currentTime);
}

export default useCurrent
```

LiveClock.jsx

```
import React from 'react'
import useCurrent from './useCurrent'

const LiveClock = () => {
  const currentTime = useCurrent();
  return (
    <div>
      <h1>Live Time</h1>
      <h1>{currentTime}</h1>
    </div>
  )
}

export default LiveClock
```

App.jsx

```
import LiveClock from './component/LiveClock'

const App = () => {
  return (
```

```

    <div>
      <LiveClock />
    </div>
  )
}

export default App

```

2. Refactor a component that fetches and displays trending movies from the TMDb API (https://api.themoviedb.org/3/trending/movie/day?api_key=YOUR_API_KEY) by moving the fetch logic into a custom hook named `useTrendingMovies`, and use this hook inside your `MoviesList` component.

Hint: The hook should return loading, error, and data states.

Answer:-

```

useTrendingMovies

import { useState, useEffect } from "react";

const useTrendingMovies = () => {
  const [movies, setMovies] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    const fetchMovies = async () => {
      try {
        const response = await fetch(
          "https://api.themoviedb.org/3/trending/movie/day?api_key=YOUR_API_KEY",
        );

        if (!response.ok) {
          throw new Error("Failed to fetch movies");
        }

        const data = await response.json();

        setMovies(data.results);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };
  });
};

```

```

        fetchMovies();
    }, []);

    return {
        movies,
        loading,
        error,
    };
};

export default useTrendingMovies;

```

MovieList.jsx

```

import React from "react";
import useTrendingMovies from "../useTrendingMovies";

const MoviesList = () => {
    const { movies, loading, error } =
        useTrendingMovies();

    if (loading) return <h2>Loading...</h2>;
    if (error) return <h2>{error}</h2>;

    return (
        <div>
            <h1>Trending Movies</h1>

            {movies.map((movie) => (
                <div key={movie.id}>
                    <h3>{movie.title}</h3>
                </div>
            ))}
        </div>
    );
};

export default MoviesList;

```

App.jsx

```

import React from 'react'
import MoviesList from '../component/MoviesList'

```

```
const App = () => {
  return (
    <div>
      <MoviesList />
    </div>
  )
}

export default App
```

3. Build a useLikeButton hook that manages the like/unlike state and count for a post, then use it to add Instagram-style like functionality to a PostCard component.

Answer:-

useLikeButton.js

```
import React, { useState } from "react";

const useLikeButton = () => {
  const [liked, setLiked] = useState(false);
  const [likes, setLikes] = useState(0);
  const toggleLike = () => {
    if (liked) {
      setLikes(likes - 1);
    } else {
      setLikes(likes + 1);
    }
    setLiked(!liked);
  };
  return {
    liked,
    likes,
    toggleLike,
  };
};

export default useLikeButton;
```

PostCard.jsx

```
import useLikeButton from './useLikeButton';

const PostCard = () => {
  const {liked, likes, toggleLike} = useLikeButton();
  return (
```

```

    <div>
      <img src="" alt="Post" />
      <h3>{likes} likes</h3>
      <button onClick={toggleLike}>
        {liked ? "Unlike" : "Like"}
      </button>
    </div>
  )
}

export default PostCard

```

App.jsx

```

import React from 'react'
import PostCard from './component/PostCard'

const App = () => {
  return (
    <div>
      <PostCard />
    </div>
  )
}

export default App

```

4. Given a component that fetches and displays Spotify playlists, refactor it to use a custom hook called useFetchData that accepts a URL and returns data, loading, and error. Replace the old fetch logic in the component with this hook.

Constraint: Do not call your custom hook conditionally or inside loops.

Answer:-

useFetchData.js

```

import React, { useEffect, useState } from 'react'

const useFetchData = (url) => {
  const [data, setData] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(()=>{
    const fetchData = async () =>{
      try {
        const response = await fetch(url);

```

```

        if(!response.ok){throw new Error("Failed to load Data");}
        const result = await response.json();
        setData(result);
    } catch (error) {
        setError(error.message);
    }
    finally{
        setIsLoading(false);
    }
}
fetchData();
},[url])

return (
    {isLoading, error, data}
)
}

export default useFetchData

```

SpotifyPlaylist.jsx

```

import useFetchData from '../component/useFetchData';

const SpotifyPlayList = () => {
    const {isLoading, error, data} =
useFetchData("https://api.spotify.com/v1/playlists");
    if(isLoading){return <h2>Loading....</h2>}
    if(error){return <h2>Error : {error}</h2>}
    return (
        <div>
            <h1>Spotify Playlist</h1>
            {data.item ?.map((playlist)=>{
                <div key={playlist.id}>
                    <h3>{playlist.name}</h3>
                </div>
            })}
        </div>
    )
}

export default SpotifyPlayList

```

App.jsx

```

import React from 'react'
import SpotifyPlaylists from '../component/SpotifyPlayList'

```

```
const App = () => {  
  return (  
    <div>  
      <SpotifyPlaylists />  
    </div>  
  )  
}  
  
export default App
```

5. Explain in your own words how creating a custom hook like useFetchData can help you reuse code across multiple components in a Flipkart-style product listing app. Give two specific examples where this hook could be used.

Answer: A custom hook like useFetchData helps us reuse the same API fetching logic across multiple components instead of writing useState, useEffect, loading, and error handling code repeatedly.

By creating the fetch logic once inside the custom hook, we can keep our components clean, readable, and easier to maintain. If we need to update the fetching logic later, we only need to modify the hook instead of changing multiple components.

Example 1: Product Listing Page

In a Flipkart-style application, useFetchData can be used to fetch and display products.

```
const { data, loading, error } = useFetchData(  
  "https://api.flipkart.com/products" );
```

This data can be used to display:

- Product Name
- Price
- Rating
- Product Image

Example 2: Categories Page

The same hook can be reused to fetch product categories.

```
const { data, loading, error } = useFetchData( "https://api.flipkart.com/categories");
```

This data can be used to display:

- Mobiles
- Electronics
- Fashion
- Furniture
- Grocery